

02_within_condition

June 24, 2026

1 Step 2: Within-Condition Benchmarking (Authoritative v4)

This notebook keeps the richer repository-facing Step 2 figure surface while treating **subject-level macro-F1** as the authoritative ranking endpoint. The within-condition benchmark matters because Step 3 transfer is only interpretable if the source-condition classifiers are strong, nested correctly, and leakage-controlled to begin with.

```
[6]: import json
import sys
from pathlib import Path

import matplotlib.colors as mcolors
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from IPython.display import display
from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix

def find_repo_root() -> Path:
    candidates = [Path.cwd().resolve(), *Path.cwd().resolve().parents]
    for base in candidates:
        if (base / 'data' / 'processed' / 'v4' / 'v4_protocol_manifest.json').
            exists():
                return base
        raise RuntimeError('Could not locate repository root from the current_
            working directory.')

REPO_ROOT = find_repo_root()
if str(REPO_ROOT) not in sys.path:
    sys.path.insert(0, str(REPO_ROOT))
if str(REPO_ROOT / 'src') not in sys.path:
    sys.path.insert(0, str(REPO_ROOT / 'src'))
STEP_TAG = 'step2'
FIG_PDF = REPO_ROOT / 'report' / 'figures' / 'v4' / 'pdf' / STEP_TAG
FIG_PNG = REPO_ROOT / 'report' / 'figures' / 'v4' / 'png' / STEP_TAG
```

```

TABLE_DIR = REPO_ROOT / 'report' / 'tables' / 'v4'
for directory in (FIG_PDF, FIG_PNG, TABLE_DIR):
    directory.mkdir(parents=True, exist_ok=True)

plt.rcParams.update({
    'font.family': 'DejaVu Serif',
    'font.size': 9,
    'axes.labelsize': 9,
    'axes.titlesize': 9,
    'xtick.labelsize': 8,
    'ytick.labelsize': 8,
    'legend.fontsize': 7,
    'figure.dpi': 180,
    'savefig.bbox': 'tight',
    'pdf.fonttype': 42,
    'ps.fonttype': 42,
})

IEEE_ONE_COL = 3.5
IEEE_TWO_COL = 7.16
COND_ORDER = ['pd', 'hd', 'als']
COND_LABEL = {'pd': 'PD', 'hd': 'HD', 'als': 'ALS'}
COND_COLOR = {'pd': '#4C78A8', 'hd': '#F58518', 'als': '#54A24B'}
CLF_ORDER = ['rf', 'knn', 'svm', 'dt', 'qda', 'xgb', 'lgbm']
CLF_LABEL = {'rf': 'RF', 'knn': 'KNN', 'svm': 'SVM', 'dt': 'DT', 'qda': 'QDA',
    ↪ 'xgb': 'XGBoost', 'lgbm': 'LightGBM'}

def savefig_stem(fig: plt.Figure, stem: str) -> None:
    pdf_path = FIG_PDF / f'{stem}.pdf'
    png_path = FIG_PNG / f'{stem}.png'
    fig.savefig(pdf_path)
    fig.savefig(png_path, dpi=300)
    print(f'Saved figure: {pdf_path.relative_to(REPO_ROOT)}')
    print(f'Saved figure: {png_path.relative_to(REPO_ROOT)}')

def save_table_csv(df: pd.DataFrame, name: str, *, index: bool = False) -> None:
    path = TABLE_DIR / name
    df.to_csv(path, index=index)
    print(f'Saved table: {path.relative_to(REPO_ROOT)}')

def ece_equal_width(y_true, prob, n_bins=10):
    y_true = np.asarray(y_true, dtype=int)
    prob = np.asarray(prob, dtype=float)
    bins = np.linspace(0.0, 1.0, n_bins + 1)

```

```

ece = 0.0
for lo, hi in zip(bins[:-1], bins[1:]):
    mask = (prob >= lo) & (prob < hi if hi < 1.0 else prob <= hi)
    if not mask.any():
        continue
    conf = prob[mask].mean()
    acc = y_true[mask].mean()
    ece += mask.mean() * abs(conf - acc)
return float(ece)

BASE = REPO_ROOT / 'experiments' / 'results' / 'v4'
within_results = {condition: json.loads((BASE / f'{condition}_results_v4.json').
    ↪read_text()) for condition in COND_ORDER}

rows = []
modal_rows = []
for condition, payload in within_results.items():
    n_folds = payload['pool_subjects']
    for clf_name, entry in payload['classifiers'].items():
        subject_metrics = entry['subject_metrics']
        subject_y = np.asarray(subject_metrics['y_true'], dtype=int)
        subject_prob = np.asarray(subject_metrics['y_prob_mean'], dtype=float)
        rows.append({
            'condition': condition,
            'classifier': clf_name,
            'subject_f1': float(entry['subject_primary_f1_macro']),
            'stride_f1': float(entry['f1_macro']),
            'subject_ci_low': float(entry['subject_primary_f1_ci_lower']),
            'subject_ci_high': float(entry['subject_primary_f1_ci_upper']),
            'stride_ci_low': float(entry['f1_macro_ci_lower']),
            'stride_ci_high': float(entry['f1_macro_ci_upper']),
            'subject_log_loss': float(subject_metrics['subject_log_loss']),
            'subject_brier': float(np.mean((subject_prob - subject_y) ** 2)),
            'subject_ece10': float(ece_equal_width(subject_y, subject_prob)),
            'specificity_control': float(subject_metrics['recall_control']),
            'recall_disease': float(subject_metrics['recall_disease']),
            'selected_strategy': entry.get('selected_imbalance_strategy', entry.
    ↪get('selected_resampling')),
            'modal_frequency': int(entry['modal_frequency']),
            'pool_subjects': int(n_folds),
        })
        params_str = ', '.join(f"{k.replace('clf__', '')}={v}" for k, v in
    ↪entry['modal_params'].items())
        modal_rows.append({
            'Condition': COND_LABEL[condition],
            'Classifier': CLF_LABEL[clf_name],

```

```

        'Selected strategy': entry.get('selected_imbalance_strategy', entry.
↳get('selected_resampling')),
        'Modal params': params_str,
        'Modal frequency': f"{entry['modal_frequency']}/{n_folds}",
    })

summary_df = pd.DataFrame(rows)
modal_df = pd.DataFrame(modal_rows)
save_table_csv(summary_df, 'step2_within_condition_summary_v4.csv')
save_table_csv(modal_df, 'step2_modal_params_v4.csv')

leader_df = (
    summary_df
    .sort_values(['condition', 'subject_f1', 'stride_f1'], ascending=[True,
↳False, False])
    .groupby('condition')
    .head(1)
    .copy()
)
leader_df['condition_label'] = leader_df['condition'].map(COND_LABEL)
leader_df['classifier_label'] = leader_df['classifier'].map(CLF_LABEL)
best_classifier = {row.condition: row.classifier for row in leader_df.
↳itertuples()}

```

Saved table: report/tables/v4/step2_within_condition_summary_v4.csv

Saved table: report/tables/v4/step2_modal_params_v4.csv

1.1 Primary overview: subject-level F1 heatmap and modal-frequency stability

The full heatmap remains the clearest subject-primary overview of Step 2. Three patterns stand out. **ALS** is the strongest source condition at the subject level, with **QDA**, **KNN**, and **SVM** all reaching perfect subject-level macro-F1 and **QDA** winning the stride-level tie-break. **HD** is most cleanly led by **XGBoost** at **0.9571** subject-level macro-F1. **PD** is flatter: **DT**, **LightGBM**, and **RF** cluster near the top, which argues against overclaiming a uniquely dominant PD classifier. The modal-frequency panel is supportive rather than primary; it is useful because it shows how stable the selected imbalance strategy is across outer folds.

```

[7]: subject_values = np.array([
    [within_results[cond]['classifiers'][clf]['subject_primary_f1_macro'] for
↳cond in COND_ORDER
    for clf in CLF_ORDER
])
freq_values = np.array([
    [within_results[cond]['classifiers'][clf]['modal_frequency'] /
↳within_results[cond]['pool_subjects'] for cond in COND_ORDER
    for clf in CLF_ORDER
])

```

```

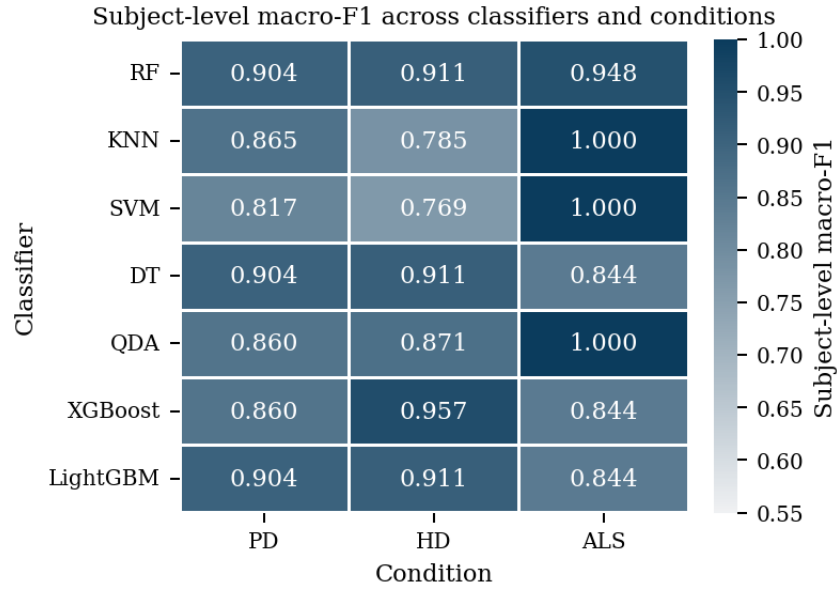
subject_df = pd.DataFrame(subject_values, index=[CLF_LABEL[c] for c in CLF_ORDER],
    columns=[COND_LABEL[c] for c in COND_ORDER])
freq_df = pd.DataFrame(freq_values, index=[CLF_LABEL[c] for c in CLF_ORDER],
    columns=[COND_LABEL[c] for c in COND_ORDER])

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.25, 3.1),
    constrained_layout=True)
sns.heatmap(
    subject_df,
    cmap=sns.light_palette('#0B3C5D', as_cmap=True),
    vmin=0.55,
    vmax=1.0,
    annot=True,
    fmt='.3f',
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Subject-level macro-F1'},
    ax=ax,
)
ax.set_title('Subject-level macro-F1 across classifiers and conditions')
ax.set_xlabel('Condition')
ax.set_ylabel('Classifier')
savefig_stem(fig, 'f1_within_condition_heatmap')
plt.show()

fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.25, 3.1),
    constrained_layout=True)
sns.heatmap(
    freq_df,
    cmap=sns.light_palette('#7A0019', as_cmap=True),
    vmin=0.0,
    vmax=1.0,
    annot=True,
    fmt='.2f',
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Modal frequency / outer folds'},
    ax=ax,
)
ax.set_title('Modal winning-frequency stability across outer folds')
ax.set_xlabel('Condition')
ax.set_ylabel('Classifier')
savefig_stem(fig, 'step2_modal_frequency_heatmap')
plt.show()

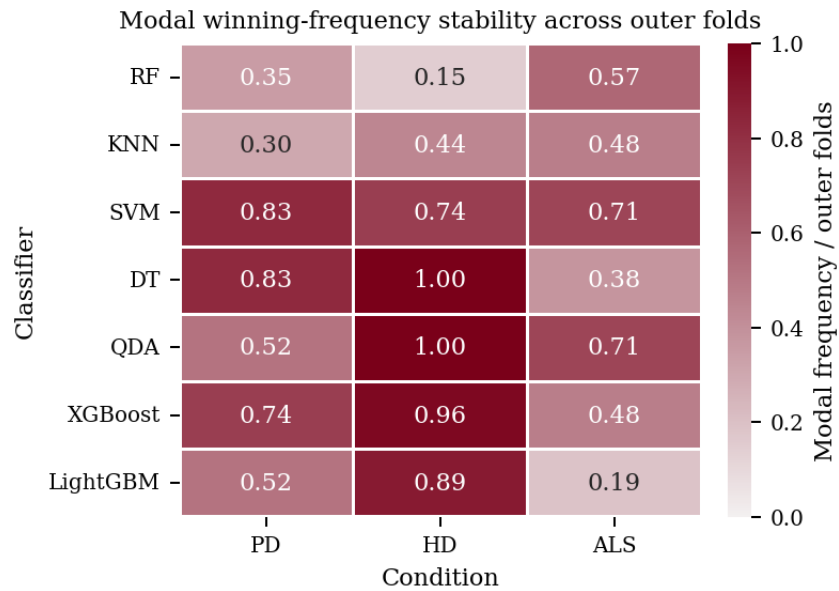
```

Saved figure: report/figures/v4/pdf/step2/f1_within_condition_heatmap.pdf
 Saved figure: report/figures/v4/png/step2/f1_within_condition_heatmap.png



Saved figure: report/figures/v4/pdf/step2/step2_modal_frequency_heatmap.pdf

Saved figure: report/figures/v4/png/step2/step2_modal_frequency_heatmap.png



1.2 Leader summary table

The leader table is where the heatmap turns into exact manuscript language. Under the authoritative subject-primary ranking, **PD** is led by **DT** (0.9042 subject F1), **HD** by **XGBoost** (0.9571),

and **ALS** by **QDA** (1.0000). The stride-level companions help resolve ties and show that the leaders are not ranking artifacts of a single metric, but the table should still be interpreted carefully: these are winners **within the repaired nested procedure**, not globally optimal classifiers outside this protocol.

```
[8]: leader_table = (
    leader_df[['condition_label', 'classifier_label', 'subject_f1',
    ↪ 'stride_f1', 'subject_log_loss', 'selected_strategy']]
    .rename(columns={
        'condition_label': 'Condition',
        'classifier_label': 'Leader',
        'subject_f1': 'Subject F1',
        'stride_f1': 'Stride F1',
        'subject_log_loss': 'Subject log loss',
        'selected_strategy': 'Strategy',
    })
    .copy()
)
leader_table[['Subject F1', 'Stride F1', 'Subject log loss']] =
    ↪ leader_table[['Subject F1', 'Stride F1', 'Subject log loss']].round(4)
save_table_csv(leader_table, 'step2_subject_stride_leaders_v4.csv')

fig, ax = plt.subplots(figsize=(IEEE_TWO_COL * 1.08, 2.35))
ax.axis('off')
cell_text = leader_table.astype(str).values.tolist()
col_widths = [0.15, 0.18, 0.15, 0.15, 0.21, 0.16]
table = ax.table(
    cellText=cell_text,
    colLabels=leader_table.columns.tolist(),
    cellLoc='center',
    colLoc='center',
    colWidths=col_widths,
    bbox=[0.01, 0.08, 0.98, 0.76],
)
table.auto_set_font_size(False)
table.set_fontsize(8.4)
table.scale(1.14, 1.58)
for (row, col), cell in table.get_celld().items():
    cell.set_edgecolor('#D0D3D4')
    cell.set_linewidth(0.6)
    if row == 0:
        cell.set_facecolor('#EAF2F8')
        cell.set_text_props(weight='bold', color='#1B2631')
    else:
        cell.set_facecolor('FFFFFF' if row % 2 else '#F8F9F9')
fig.suptitle('Subject-primary leaders by condition', y=0.96, fontsize=10)
savefig_stem(fig, 'step2_subject_stride_leaders')
```

```

plt.show()

strategy_counts = summary_df.groupby(['condition', 'selected_strategy']).size().
    ↪reset_index(name='count')
fig, ax = plt.subplots(figsize=(IEEE_ONE_COL * 1.25, 2.9),
    ↪constrained_layout=True)
x = np.arange(len(COND_ORDER))
width = 0.24
for offset, strategy, color in [
    (-width, 'raw', '#4C78A8'),
    (0.0, 'balanced', '#9EC5E5'),
    (width, 'synthetic', '#C44E52'),
]:
    vals = [int(strategy_counts[(strategy_counts['condition'] == cond) &
    ↪(strategy_counts['selected_strategy'] == strategy)]['count'].sum()) for cond,
    ↪in COND_ORDER]
    bars = ax.bar(x + offset, vals, width=width, label=strategy, color=color,
    ↪edgecolor='black', linewidth=0.4)
    for bar in bars:
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.04,
        ↪f'{int(bar.get_height())}', ha='center', va='bottom', fontsize=8)
ax.set_xticks(x)
ax.set_xticklabels([COND_LABEL[c] for c in COND_ORDER])
ax.set_ylabel('Selected classifiers')
ax.set_title('Selected imbalance-strategy mix')
ax.grid(axis='y', alpha=0.2)
ax.legend(loc='center left', bbox_to_anchor=(1.02, 0.5), frameon=False)
savefig_stem(fig, 'step2_strategy_summary')
plt.show()

```

Saved table: report/tables/v4/step2_subject_stride_leaders_v4.csv

Saved figure: report/figures/v4/pdf/step2/step2_subject_stride_leaders.pdf

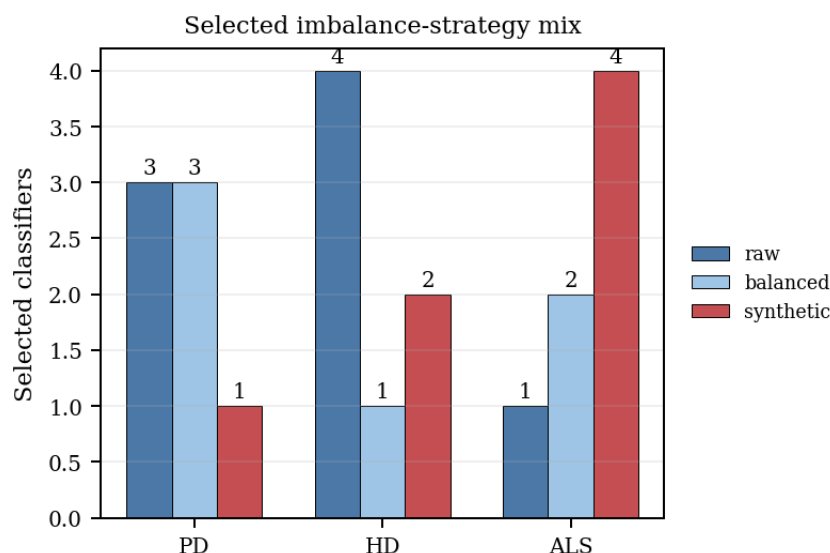
Saved figure: report/figures/v4/png/step2/step2_subject_stride_leaders.png

Subject-primary leaders by condition

Condition	Leader	Subject F1	Stride F1	Subject log loss	Strategy
ALS	QDA	1.0	0.9642	0.0481	synthetic
HD	XGBoost	0.9571	0.9546	0.1524	raw
PD	DT	0.9042	0.9022	1.2065	balanced

Saved figure: report/figures/v4/pdf/step2/step2_strategy_summary.pdf

Saved figure: report/figures/v4/png/step2/step2_strategy_summary.png



1.3 Styled tables: leaders, modal settings, and descriptive probability diagnostics

These tables keep the repository notebook more informative than the paper alone. The leader and modal-parameter tables show how classifier family and selected imbalance strategy vary by condition, while the descriptive probability diagnostics show that strong classification does not imply identical probability behavior. In particular, the **PD DT leader** combines a strong subject-level F1 with noticeably less favorable subject log loss than the **HD** and **ALS** leaders, which is exactly why calibration-style quantities should remain companion diagnostics rather than the primary ranking rule. Because the subject counts are modest, metrics such as ECE should be described cautiously and not overinterpreted as precise calibration guarantees.

```
[9]: leader_view = leader_df[['condition_label', 'classifier_label', 'subject_f1',  
    ↳ 'stride_f1', 'subject_log_loss', 'subject_brier', 'subject_ece10',  
    ↳ 'specificity_control', 'recall_disease', 'selected_strategy']].  
    ↳ rename(columns={  
        'condition_label': 'Condition',  
        'classifier_label': 'Leader',  
        'subject_f1': 'Subject F1',  
        'stride_f1': 'Stride F1',  
        'subject_log_loss': 'Subject log loss',  
        'subject_brier': 'Subject Brier',  
        'subject_ece10': 'Subject ECE (10 bins)',  
        'specificity_control': 'Control specificity',  
        'recall_disease': 'Disease recall',
```

```

        'selected_strategy': 'Selected strategy',
    })

display(
    leader_view.style
    .format({
        'Subject F1': '{:.4f}',
        'Stride F1': '{:.4f}',
        'Subject log loss': '{:.4f}',
        'Subject Brier': '{:.4f}',
        'Subject ECE (10 bins)': '{:.4f}',
        'Control specificity': '{:.4f}',
        'Disease recall': '{:.4f}',
    })
    .background_gradient(subset=['Subject F1'], cmap='Blues', vmin=0.55, vmax=1.
    ↪0)
    .set_caption('Best classifier in each condition under the subject-primary_
    ↪endpoint')
)

display(
    modal_df.style
    .set_caption('Selected modal parameters and selected-strategy frequencies_
    ↪across outer folds')
)

```

```
<pandas.io.formats.style.Styler at 0x7f5eb1f532c0>
```

```
<pandas.io.formats.style.Styler at 0x7f5eb1f532c0>
```

1.4 Confusion matrices for the subject-primary leaders

These confusion matrices translate the leaderboard into error structure. They are useful because they show whether a selected leader is balancing **control specificity** and **disease recall** rather than succeeding by inflating only one side of the binary task. They also make the unit of analysis tangible: the benchmark is evaluated at the held-out **subject** level, not at the raw stride-row level. The confusion matrices therefore support the subject-primary ranking, but they do not replace it.

```

[10]: fig, axes = plt.subplots(1, 3, figsize=(IEEE_TWO_COL * 0.98, 3.0),
    ↪constrained_layout=True)
for idx, (ax, condition) in enumerate(zip(axes, COND_ORDER)):
    clf_name = best_classifier[condition]
    subject_metrics =
    ↪within_results[condition]['classifiers'][clf_name]['subject_metrics']
    y_true = subject_metrics['y_true']
    y_pred = subject_metrics['y_pred']
    cm = confusion_matrix(y_true, y_pred, normalize='true')

```

```

    disp = ConfusionMatrixDisplay(cm, display_labels=['Control', 'PD',
↪COND_LABEL[condition]])
    disp.plot(ax=ax, cmap='Blues', colorbar=False, values_format='.2f')
    ax.set_title(f"{COND_LABEL[condition]} / {CLF_LABEL[clf_name]}", fontsize=8.
↪5, pad=6)
    if idx != 0:
        ax.set_ylabel('')
    ax.grid(False)
fig.suptitle('Subject-level normalized confusion matrices for the_
↪subject-primary leaders', y=1.02)
savefig_stem(fig, 'step2_best_classifier_confusion_matrices')
plt.show()

```

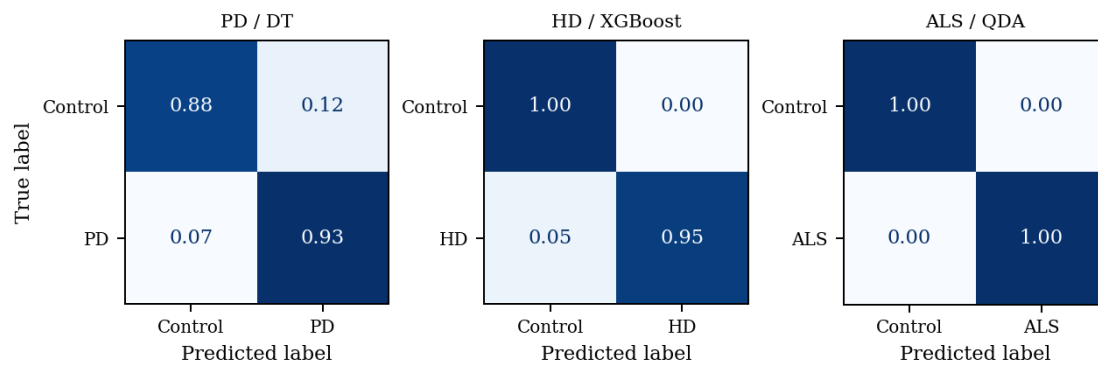
Saved figure:

report/figures/v4/pdf/step2/step2_best_classifier_confusion_matrices.pdf

Saved figure:

report/figures/v4/png/step2/step2_best_classifier_confusion_matrices.png

Subject-level normalized confusion matrices for the subject-primary leaders



1.5 Reporting guidance

The strongest within-condition summary is that **ALS is easiest under the repaired v4 protocol**, **HD has a clear XGBoost leader**, and **PD is comparatively flatter across several competitive families**. Subject-level macro-F1 remains authoritative, stride-level metrics remain descriptive companions, and the strategy selections should be reported as nested-procedure outcomes rather than as decontextualized best practices. Small cohort sizes, especially in ALS, still justify caution when translating these strong benchmarks into broader clinical claims.